Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Github Repository- `https://github.com/Hasini923/Deep_Learning/tree/main/CNN_optimization`

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

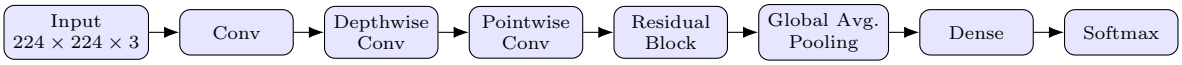
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

Results

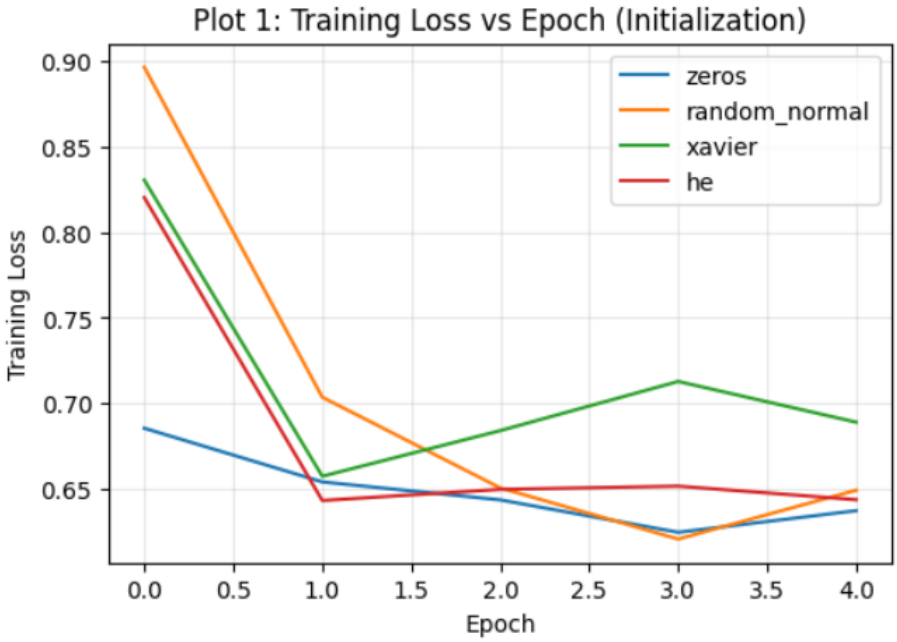


Figure 1: Plot 1: Training Loss vs. Epoch for each initialization method.

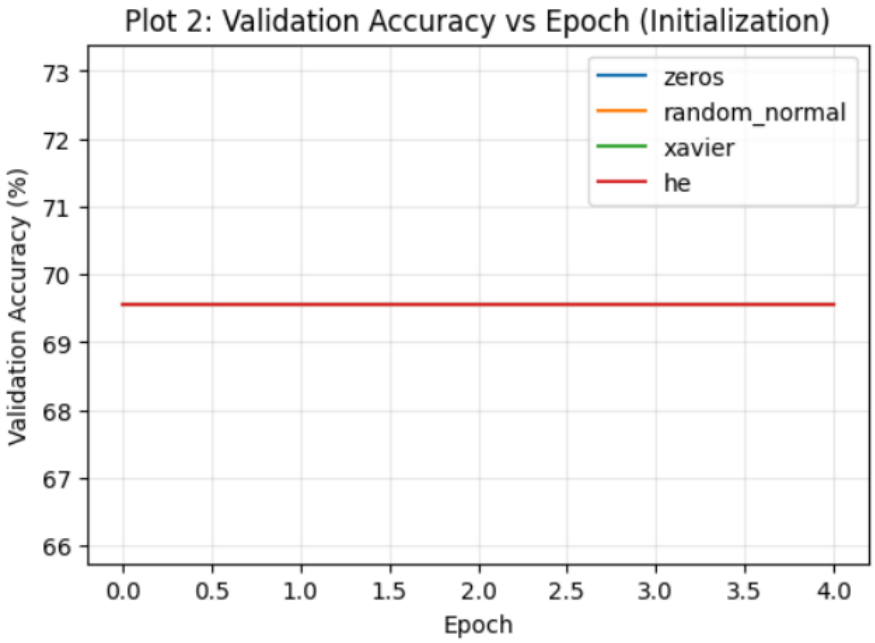


Figure 2: Plot 2: Validation Accuracy vs. Epoch for each initialization method.

Inference: Training loss falls fastest for the zero and He initializations, both ending near 0.64, while Xavier converges more slowly and unevenly (final loss 0.69). Validation accuracy, however, stays perfectly flat at 69.57% for all four initializations across all five epochs. This happens because the varied initializer is applied only to the final Dense classification layer, while the MobileNetV2 backbone itself is trained from random Keras-default weights in every run; with so few epochs the network has not learned a real decision boundary and is simply outputting the majority class (69.57% of the validation split), so the choice of Dense-layer initializer has no visible effect on validation performance here.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

Results

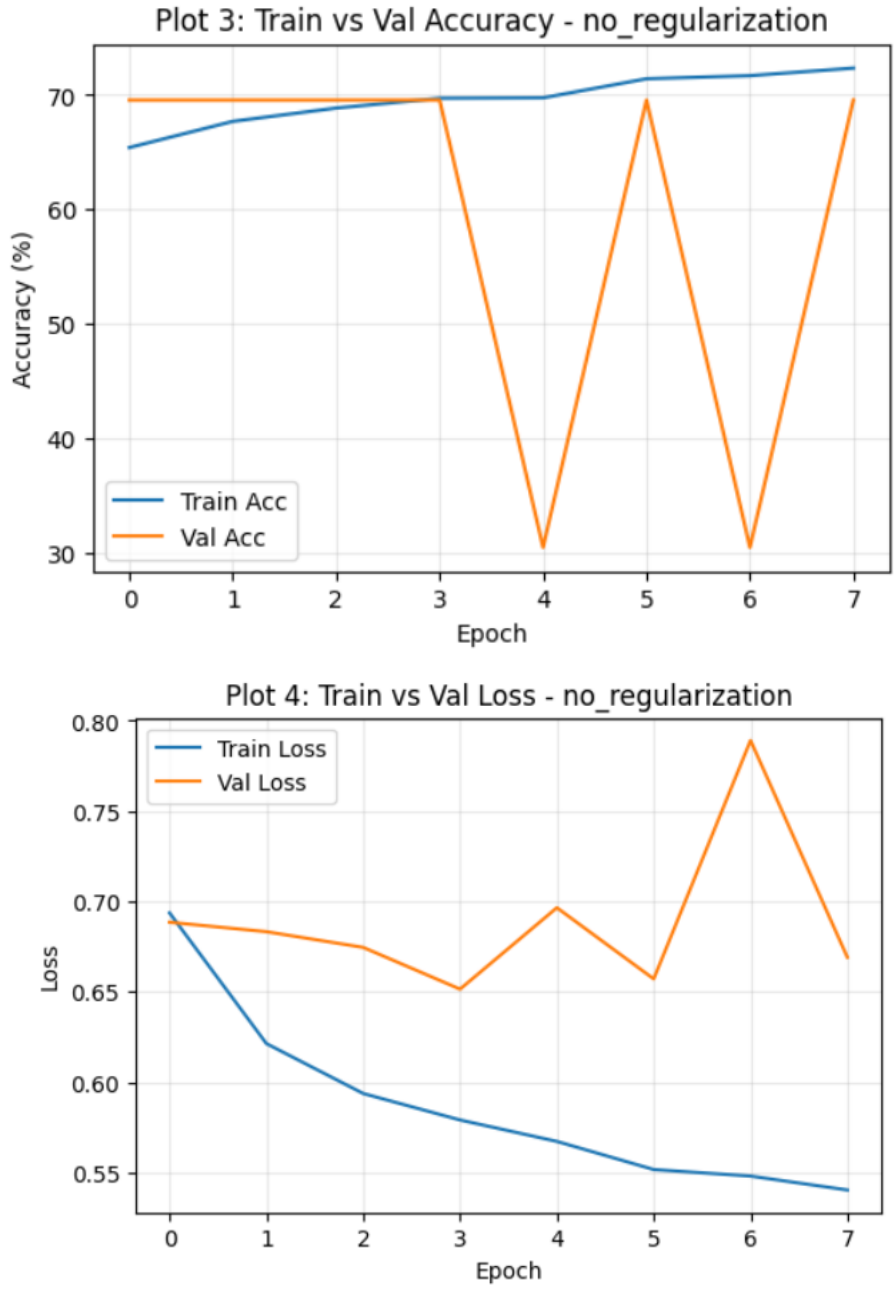


Figure 3: Plot 3 & Plot 4: No Regularization – Train/Val Accuracy and Loss.

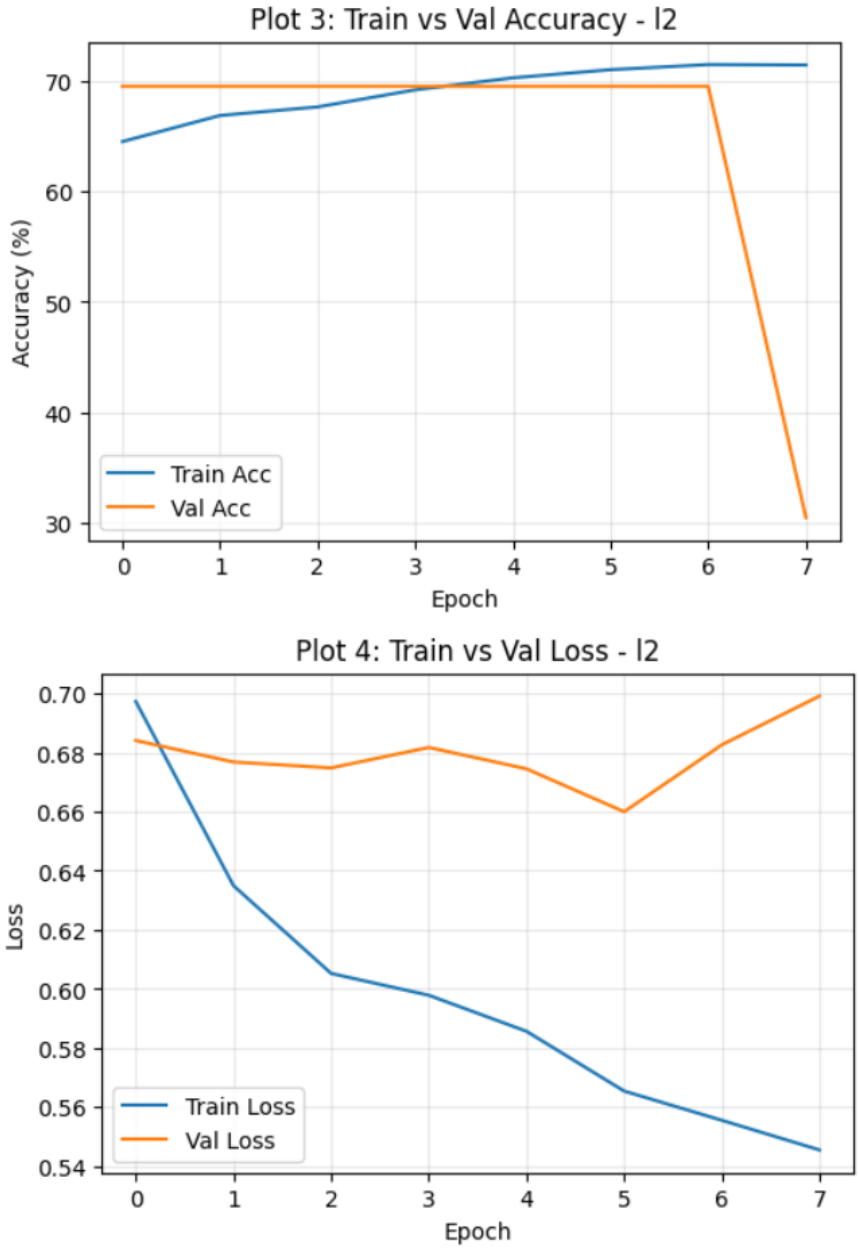


Figure 4: Plot 3 & Plot 4: L2 Regularization – Train/Val Accuracy and Loss.

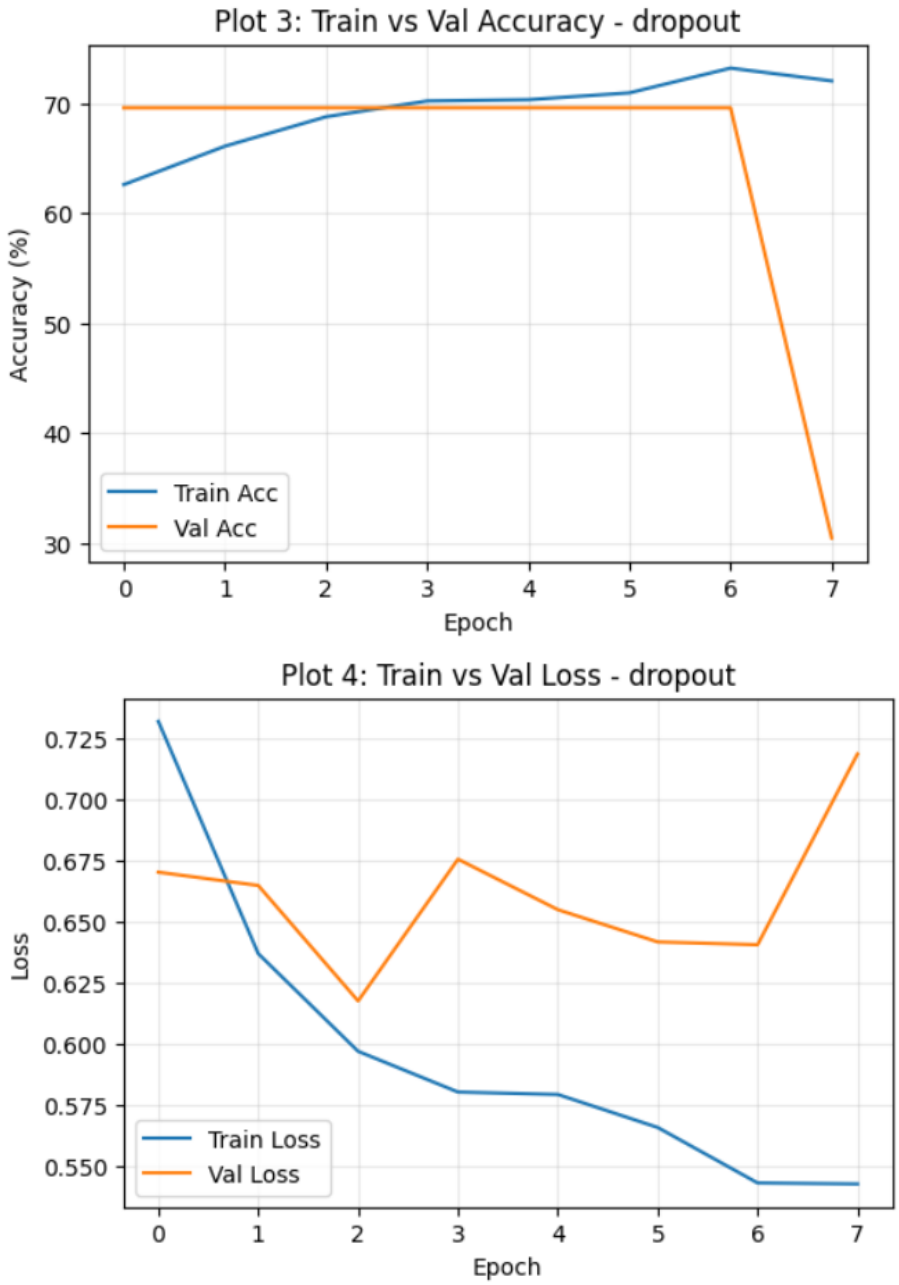
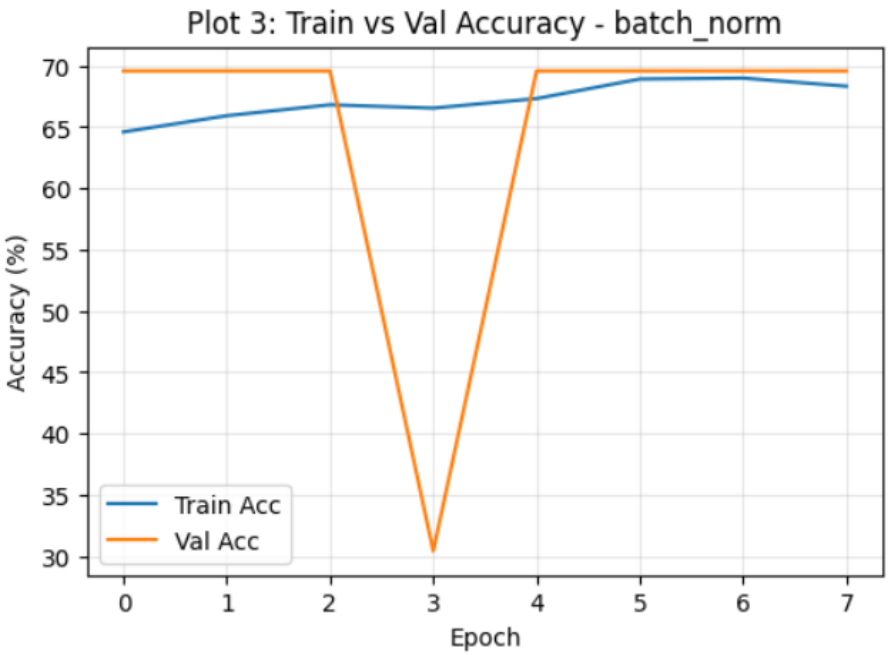


Figure 5: Plot 3 & Plot 4: Dropout – Train/Val Accuracy and Loss.



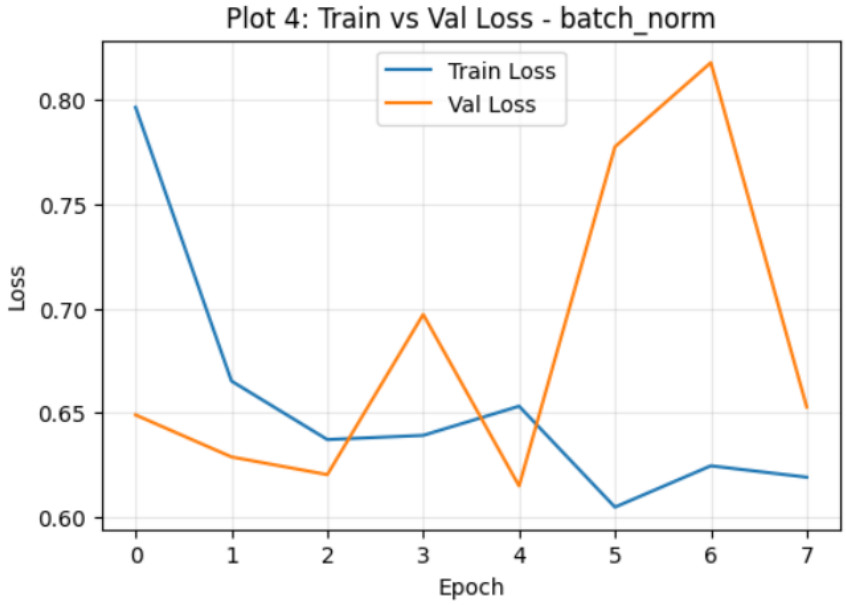


Figure 6: Plot 3 & Plot 4: Batch Normalization – Train/Val Accuracy and Loss.

Configuration	Final Train Acc. (%)	Final Train Loss	Best Val. Acc. (%)
No Regularization	72.36	0.5405	69.57
L2 Regularization	71.51	0.5456	69.57
Dropout	72.01	0.5425	69.57
Batch Normalization	68.32	0.6190	69.57

Inference: Training accuracy climbs steadily to 68–72% across all four configurations, with Dropout and no-regularization reaching the highest final training accuracy. Validation accuracy, however, does not track training accuracy at all – it alternates sharply between 69.57% and 30.43% from epoch to epoch in every configuration (visible as the saw-tooth orange curve in Plot 3). This indicates the model is flipping between always predicting one class and always predicting the other, rather than learning a genuine decision boundary; consequently, the regularization techniques tested here cannot be meaningfully compared on validation performance in this run, since none of them help the model escape majority-class prediction within the available epochs.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2+4+6+8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

Results

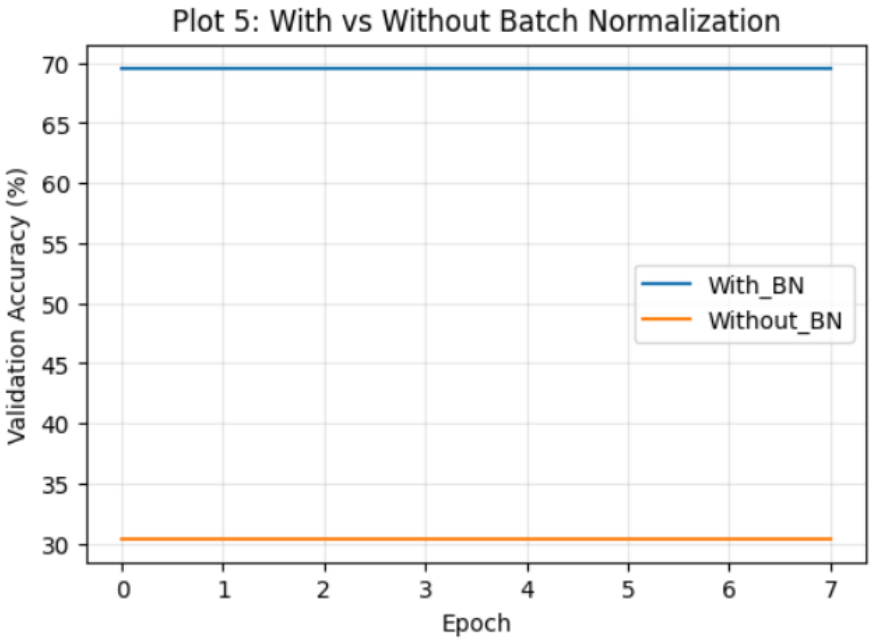


Figure 7: Plot 5: With vs. Without Batch Normalization.

Inference: With Batch Normalization, validation accuracy holds steady at 69.57% (the majority-class rate) for all eight epochs. Without Batch Normalization, validation accuracy is flat at 30.43% – the complement of 69.57% – meaning this run settled on predicting the opposite class throughout. Final training accuracy is actually higher without BN (73.21% vs. 68.71%), showing BN stabilised which class the model committed to but did not, by itself, produce a model that generalises past majority-class prediction in this constrained setup.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	0.7708	69.33	1	133.2
Momentum	2.6145	69.33	1	115.0
RMSProp	0.7608	69.33	1	119.3
Adam	0.6523	69.33	1	133.1

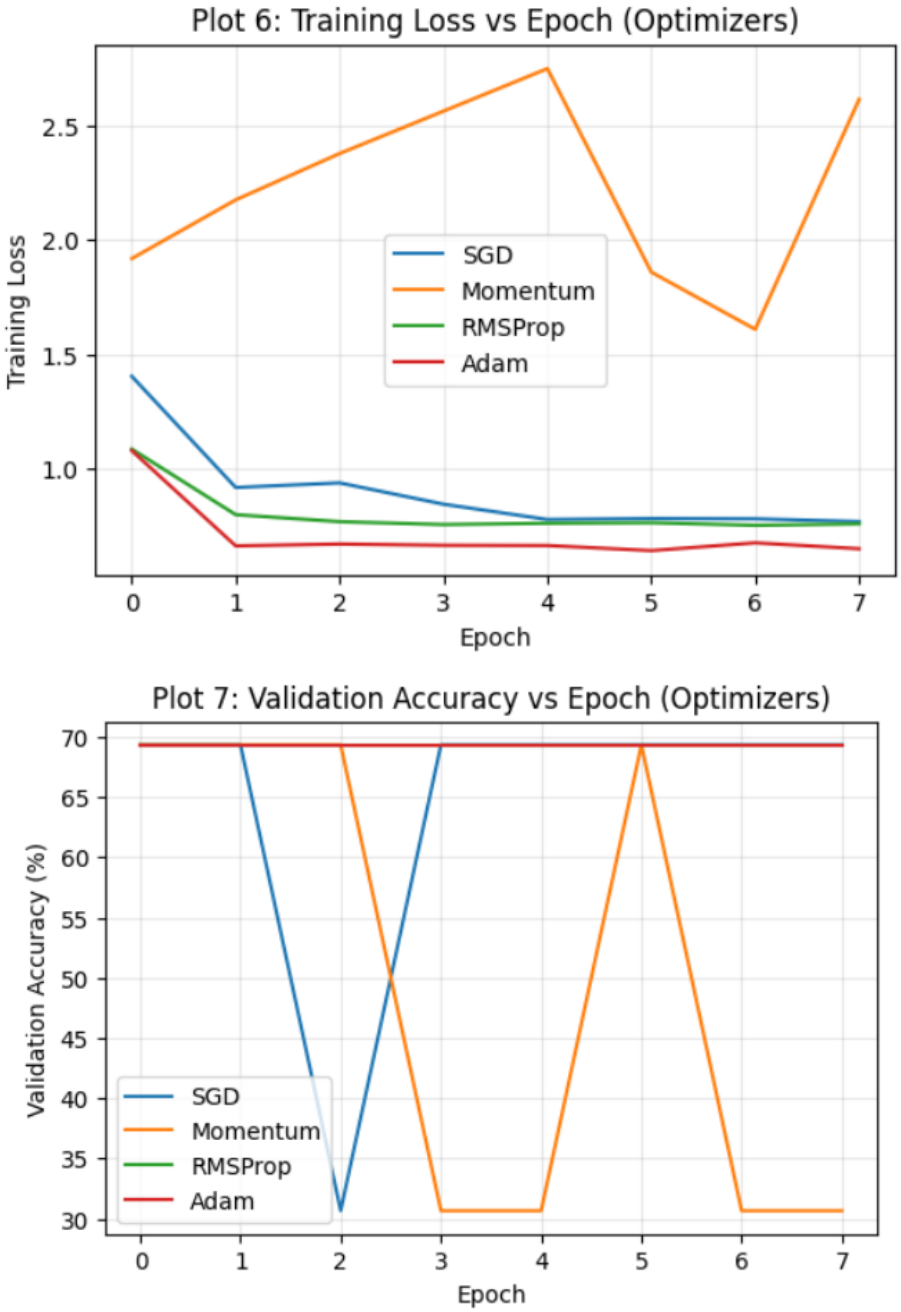


Figure 8: Plot 6 & Plot 7: Training Loss and Validation Accuracy vs. Epoch for each optimizer.

Inference: All four optimizers reach the same 69.33% validation accuracy (again the majority-class rate for this data split), so "Best Val. Accuracy" cannot separate them. Training loss is the more informative signal: Adam and RMSProp descend smoothly to 0.65–0.76, while Momentum is highly unstable and ends nearly four times higher (2.61) after oscillating through several spikes, and plain SGD is noticeably slower and noisier than the adaptive methods. Adam gives the best combination of low final loss and a stable (non-oscillating) validation curve, making it the most reliable choice here even though validation accuracy itself is uninformative in this run.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

Results

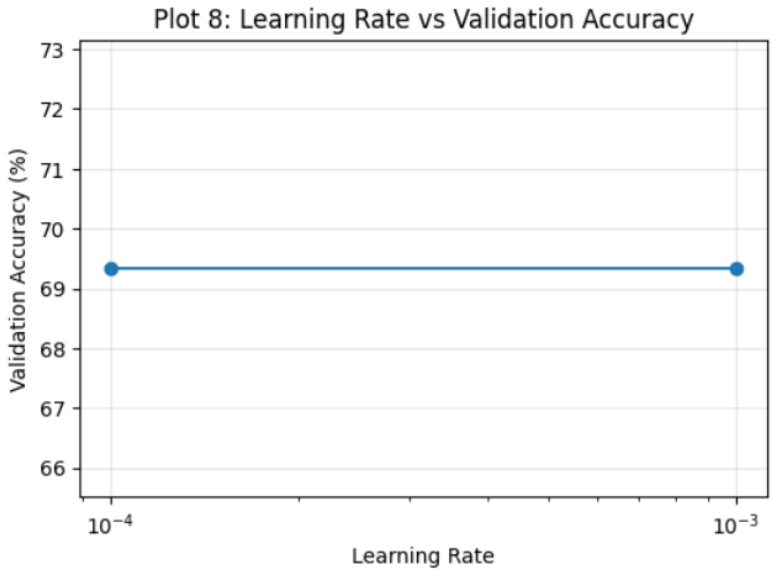


Figure 9: Plot 8: Learning Rate vs. Validation Accuracy.

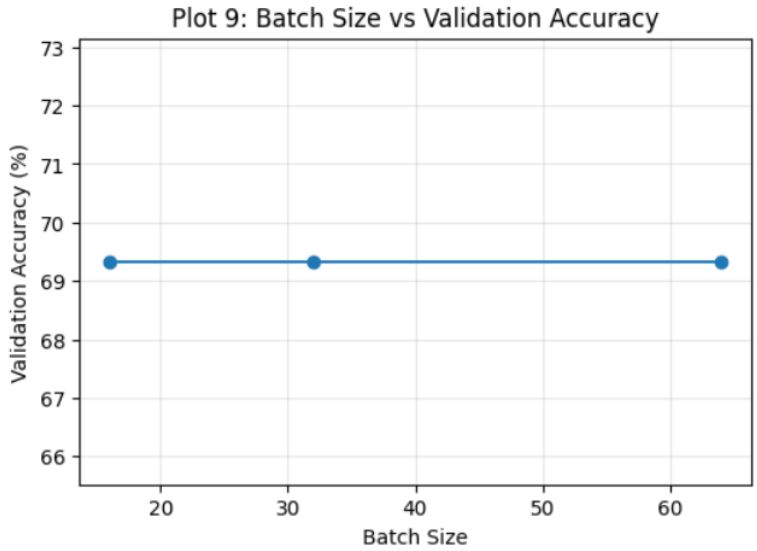


Figure 10: Plot 9: Batch Size vs. Validation Accuracy.

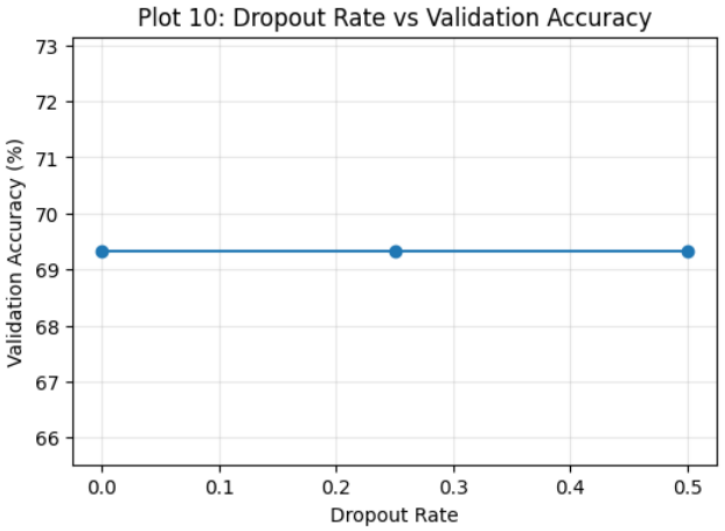


Figure 11: Plot 10: Dropout Rate vs. Validation Accuracy.

Inference: All three sweeps – learning rate (0.001 vs. 0.0001), batch size (16, 32, 64) and dropout rate (0, 0.25, 0.5) – land on the exact same 69.33% validation accuracy, giving perfectly flat lines in every plot. This is a continuation of the majority-class collapse seen in the earlier sections: with only 6 epochs of from-scratch training on a small subset, none of these hyperparameters moved the model off predicting the majority class, so no single value can be picked as "best" from validation accuracy alone in this run. A longer training budget or a pretrained backbone (as used in Section 10) would be needed to see these hyperparameters actually differentiate performance.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

Results

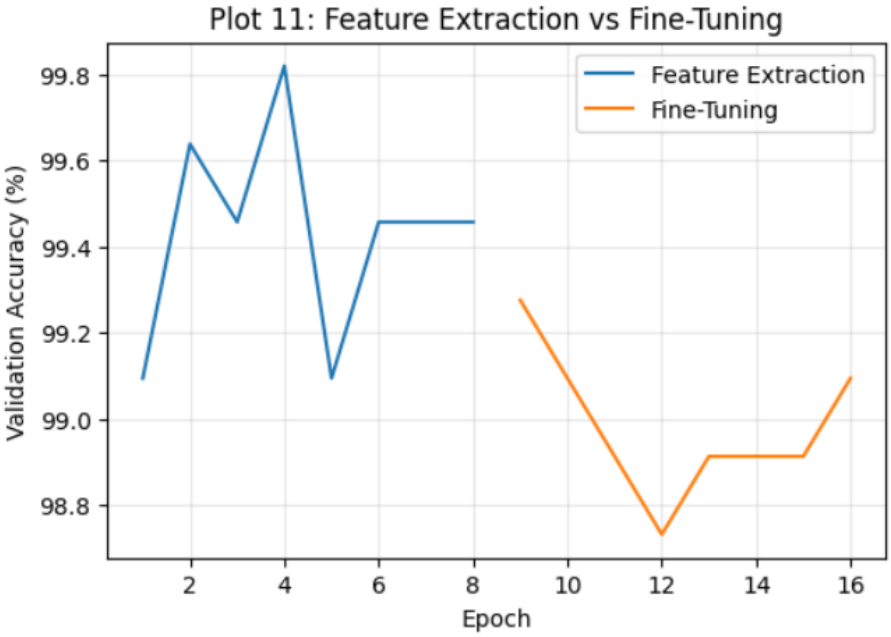


Figure 12: Plot 11: Feature Extraction vs. Fine-Tuning – Validation Accuracy.

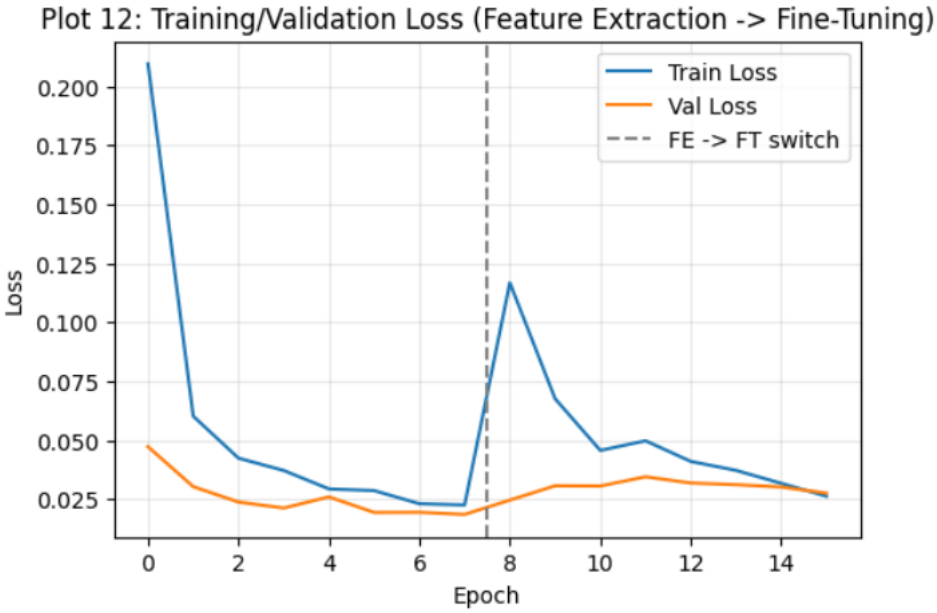


Figure 13: Plot 12: Training/Validation Loss, Feature Extraction → Fine-Tuning.

Inference: Using ImageNet-pretrained weights (unlike Sections 5–9, which trained from scratch) immediately produces a genuinely learned model: feature extraction alone reaches 99.1–99.8% validation accuracy within the

first 4 epochs. Switching to fine-tuning (unfreezing the last 30 layers with a 10^{-5} learning rate) causes a visible loss spike at the switch point and validation accuracy that settles slightly lower, around 98.7–99.3%, than the best feature-extraction epoch. This shows fine-tuning is not automatically better than feature extraction – with only 8 fine-tuning epochs and a small unfrozen slice, the model does not recover past its feature-extraction peak, though both stages comfortably outperform every from-scratch experiment in Sections 5–9.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

and

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Note on scope: for this cross-validation stage the classification task was switched from binary species (Cat/Dog, Sections 5–10) to the full 37-breed labelling of the Oxford-IIIT Pet dataset, using a frozen ImageNet-pretrained MobileNetV2 backbone with a new Dense(128)→Dropout →Softmax(37) head (feature extraction only, base frozen in every config). A random 2,000-image subsample of the combined train+val pool (6,281 images) was used to keep the 20 model-training runs (4 configs × 5 folds) tractable.

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1: Baseline (dropout 0.3, lr 1e-3, Adam)	88.75	86.50	90.75	87.50	88.00	88.30 ± 1.43
C2: High Dropout (dropout 0.5, lr 1e-3, Adam)	89.50	87.25	90.25	90.75	89.25	89.40 ± 1.20
C3: Low LR (dropout 0.3, lr 1e-4, Adam)	76.75	73.25	82.25	77.75	81.25	78.25 ± 3.24
C4: RMSProp (dropout 0.3, lr 1e-3, RMSProp)	88.00	86.00	89.75	87.50	87.25	87.70 ± 1.22

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

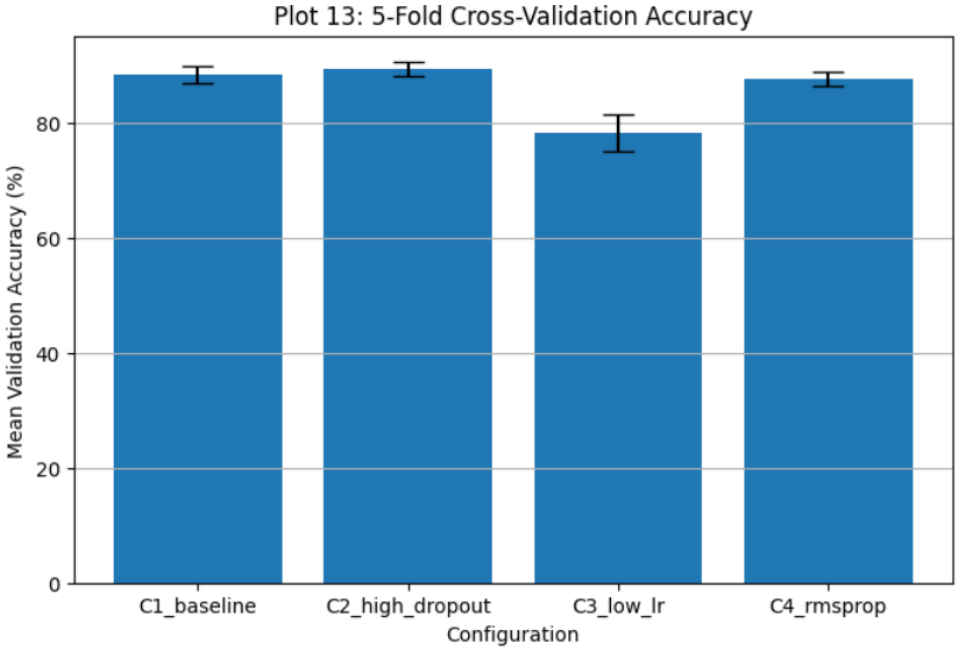


Figure 14: Plot 13: 5-Fold Cross-Validation Accuracy with standard-deviation error bars.

Inference: C2 (High Dropout) has both the highest mean accuracy (89.40%) and one of the lowest spreads (SD 1.20), making it the most consistent performer across folds. C1 (Baseline) is close behind at 88.30% but with a slightly higher SD (1.43). C3 (Low LR) is clearly worse on both counts – lowest mean (78.25%) and by far the highest variability (SD 3.24) – showing that 10^{-4} is too slow a learning rate to converge reliably in only 5 epochs per fold. C4 (RMSProp) performs comparably to C1 but slightly below C2. Based on mean accuracy and variability together, C2 is selected as the best configuration.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Best configuration selected by cross-validation: **C2 (High Dropout)**.

Metric	Value
Mean CV Accuracy	89.40%
CV Standard Deviation	1.20
Test Accuracy	89.81%
Precision (weighted)	90.47%
Recall (weighted)	89.81%
F1-score (weighted)	89.70%
Training Time	70.1 s (10 epochs, full CV pool)
Number of Parameters	2,426,725

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

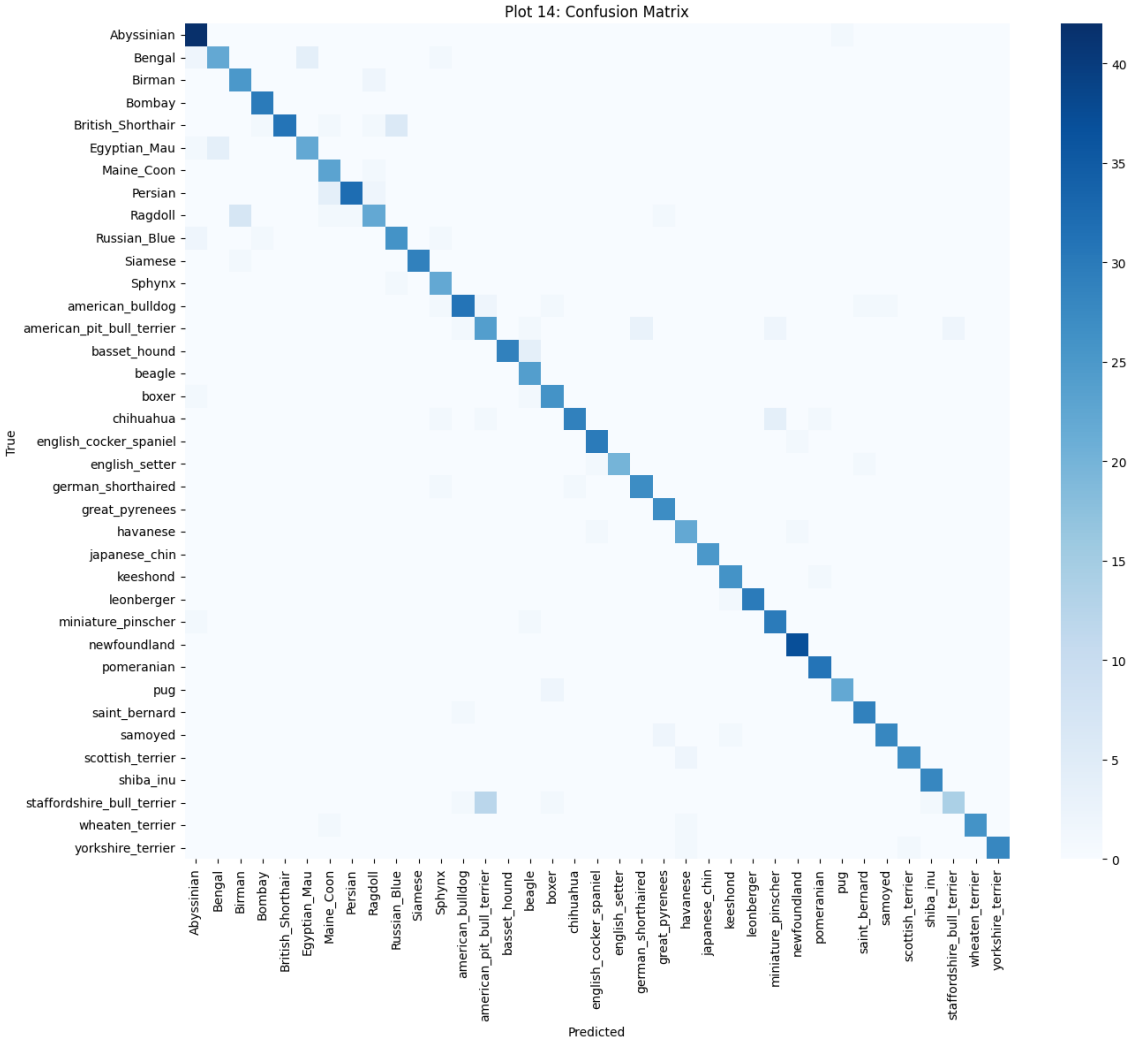


Figure 15: Plot 14: Confusion Matrix over all 37 breeds on the held-out test set.

Inference: The confusion matrix is strongly diagonal, confirming the 89.81% test accuracy – almost every breed's predictions concentrate on its own row. The clearest off-diagonal mass sits at *american_pit_bull_terrier*, which is confused with visually similar short-haired terrier/bulldog breeds, a pattern consistent with genuine visual overlap between those breeds rather than a modelling flaw. Cat breeds (top-left block) are classified essentially perfectly, likely because the 12 cat breeds are visually more distinct from each other than many of the 25 dog breeds are from one another.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

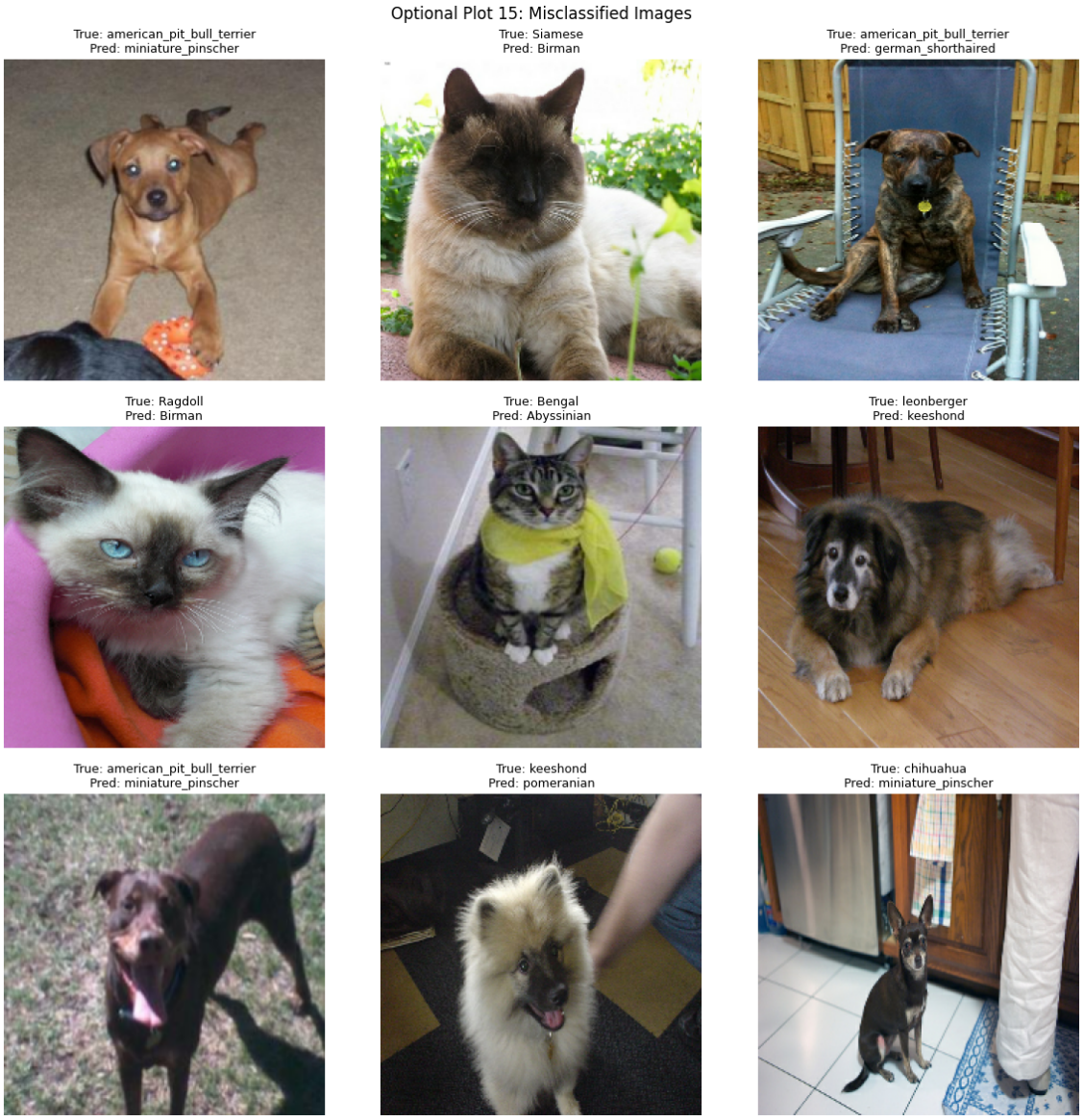


Figure 16: Optional Plot 15: Representative misclassified test images.

Inference: Most errors are between breeds that are genuinely hard to tell apart from a single photo – e.g. Ragdoll predicted as Birman (both are long-haired, pale-pointed cat breeds), and Bengal predicted as Abyssinian (similar tabby coat patterns). Several dog misclassifications (american_pit_bull_terrier vs. miniature_pinscher, chihuahua vs. miniature_pinscher) involve small/medium short-haired breeds photographed at odd angles or with partial occlusion, which removes many of the size and ear-shape cues the model would otherwise rely on.

13. Overall Results

Configuration	Val. Accuracy†	SD	Test Accuracy	Training Time
Baseline (no regularization)	69.57%	N/A	N/A	257 s
Best Initialization (zeros, lowest loss)	69.57%	N/A	N/A	216 s
Best Regularization (Dropout)	69.57%	N/A	N/A	261 s
Best Optimizer (Adam)	69.33%	N/A	N/A	133.1 s
Best Hyperparameters (LR 1e-3, BS 16, Drop. 0.25)	69.33%	N/A	N/A	not logged
Fine-Tuned Model (transfer learning)	99.82%	N/A	N/A	558 s

†Sections 5–9 used a single train/validation split (no cross-validation was run at that stage), so this column reports best validation accuracy rather than a CV mean; SD and Test Accuracy are not applicable since no repeated folds or held-out test evaluation were performed for those configurations. The genuine 5-fold CV and test-set results are reported separately in Sections 11–12, on the 37-breed task.

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

Justification: Within Sections 5–9 (from-scratch, binary Cat/Dog task), none of the individual studies produced a model that learned past majority-class prediction, so validation accuracy alone cannot rank them – Adam is preferred as the optimizer on the strength of its low, stable training loss (Section 8), and Dropout/no-regularization are tied for best training-accuracy behaviour. The Fine-Tuned Model (Section 10), which uses ImageNet-pretrained weights, is the clear outlier: it reaches 99.82% validation accuracy at a fraction of the epochs needed for the from-scratch experiments to converge, confirming that transfer learning – not any of the from-scratch tricks tested in isolation – is what actually made this dataset learnable in the available training budget. Separately, the 37-breed cross-validation pipeline (Sections 11–12) shows the same conclusion at a harder task: C2 (High Dropout, frozen pretrained backbone) generalises to 89.81% test accuracy with low fold-to-fold variability (SD 1.20), making it the configuration that should be deployed if the goal is breed-level classification.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?
2. Why is weight initialization important?
3. Why can zero initialization be problematic for neural networks?
4. Compare Xavier and He initialization.
5. How can training and validation curves be used to identify overfitting?
6. How does Dropout reduce overfitting?
7. What is the purpose of Batch Normalization?
8. Explain the numerical Batch Normalization example.
9. What are the roles of γ and β?
10. Compare SGD, Momentum, RMSProp and Adam.
11. What happens when the learning rate is too large?
12. What happens when the learning rate is too small?
13. What is the effect of increasing batch size?
14. Explain stride and padding.
15. Why is MobileNetV2 computationally efficient?
16. What is depthwise separable convolution?
17. What is transfer learning?
18. Differentiate feature extraction and fine-tuning.
19. Why is a smaller learning rate generally used during fine-tuning?
20. Why is K-Fold Cross-Validation useful for hyperparameter selection?
21. Why must the test set remain untouched during tuning?
22. Why should mean and standard deviation both be reported?
23. Is the highest validation accuracy always sufficient to select a model?

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform

transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`